Faculty of Engineering and Materials Sciences
Mechatronics Engineering Department
German University in Cairo

Place and Sort 3 DoF Robot Manupilator in a Warehouse

Submitted by

Amr Wael 52-12977

Nabil Hassan 52-11233

Ahmed Daw 52-6444

Mohamed Ashraf 52-17791

Mohamed Yasser 52-12906

Yassin Eissa 52-20855

Supervised by

Assoc. Prof.Dr. Omar Shehata

Chapter 1

INTRODUCTION

In current warehouse management, efficacy along with accuracy is essential to raising production levels and achieving some of the organizational goals. As a result, robotic automation has become a considerable asset in the achievement of such goals, with manipulators being highly utilized in sorting, organizing, and transporting goods. This project focuses on the design and implementation of a 4 Degree of Freedom (DoF) robot manipulator for sorting and placing objects within a warehouse.

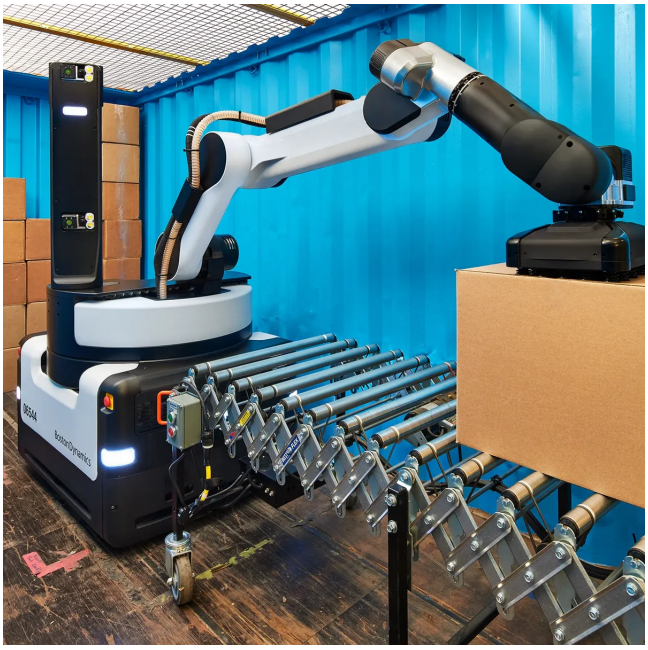

Figure 1.1: Robotic Manipulator in action

The four Degrees of Freedom robot manipulator can accomplish a number of intricate functions requiring precision and some spatial manipulation of objects thus ideal for warehouse automation. It also has an articulated arm that eases motion in different planes enabling the robotic part to pick up, carry and place objects in

specific angles. There is also adaptability of the robot, decreasing the necessity of manual work in the performing such actions that are repetitive thereby increasing both the sorting speed and accuracy.

In this project, the robot manipulator is able to identify the items, classify them according to the rules of the given system and store the items in the appropriate place. With the use of the sensors and control algorithms, the robot can move and perform tasks even in a changing environment as found in a warehouse. Because these processes would be carried out by the system, the system aims to enhance control of the inventories, manage the processes effectively, and hence increase the efficiency of warehouses operations.

This report also explains the process of designing, system construction and enunciating functions of the 4 DoF robot manipulator and describes, in general, how such a system can alter the concept of warehousing systems.

Chapter 2

DH-Convention

2.1 Denavit-Hartenberg (DH) Convention

The Denavit-Hartenberg (DH) convention is a systematic method used in robotics to describe the kinematic structure of a robot, particularly the relationships between consecutive joints. It simplifies the process of deriving forward kinematics, which involves calculating the position and orientation of the robot's end-effector relative to its base.
The DH convention uses a set of four parameters to describe each joint in a robot manipulator:

- **Link Length** (a): The distance between the axes of two consecutive joints along the common normal.
- **Link Twist** (α): The angle between the two consecutive joint axes.
- **Link Offset** (d): The distance along the previous joint axis to the common normal.
- **Joint Angle** (θ): The rotation around the current joint axis to align with the next.

By applying these parameters, the DH convention defines a transformation matrix for each joint, making it easier to model and analyze the overall motion of robotic manipulators.

2.2 Coordinate Frame Assignment

The image shows a 4 Degree of Freedom (DoF) robotic arm with multiple coordinate frames assigned at different joints, each corresponding to a part of the robot's

kinematic structure. The coordinate systems are labeled "Coordinate System 1," "Coordinate System 2," "Coordinate System 4," and so on. These frames are crucial for describing the position and orientation of each link relative to its preceding link, following the Denavit-Hartenberg (DH) convention.

- **Coordinate System 1** appears to be attached near the end-effector, showing the frame for the final link, which likely defines the position of the gripper or tool. - **Coordinate System 2** is placed at the base of the robot, representing the reference frame from which all subsequent joint transformations originate. - **Coordinate System 4** is located at the second joint, indicating the rotation axis of the arm where joint transformations will be applied.

Each frame defines the origin and orientation of the respective link, with the X, Y, and Z axes labeled to indicate directions in 3D space. The consistency in frame assignment is crucial for calculating forward and inverse kinematics, as it allows for the precise determination of the robot's end-effector position through sequential transformations across joints.

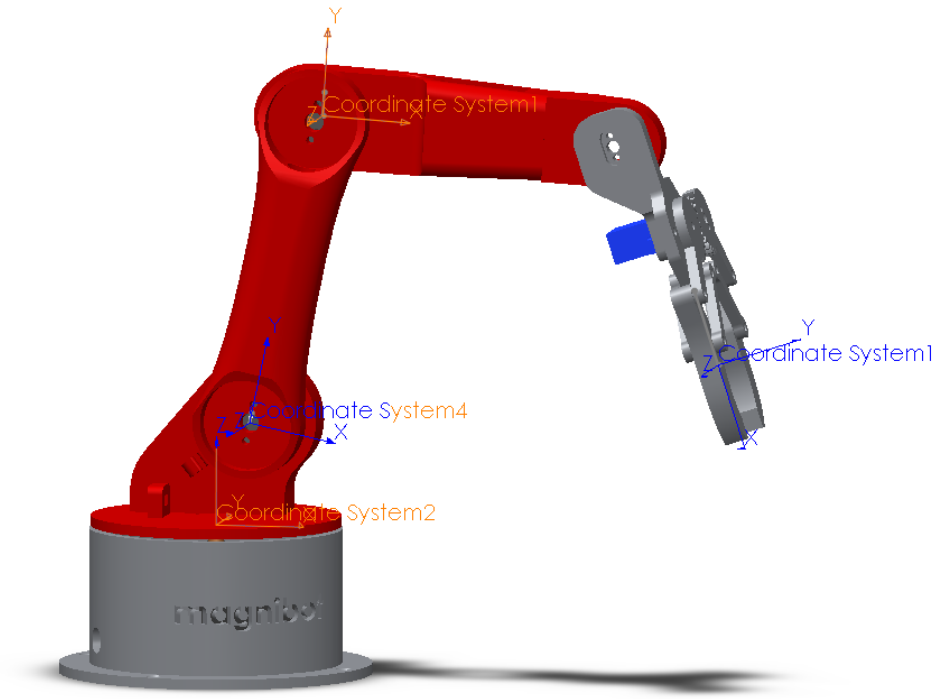

Figure 2.1: Coordinate Frame Assignment

2.3 DH-Convention of the Robot

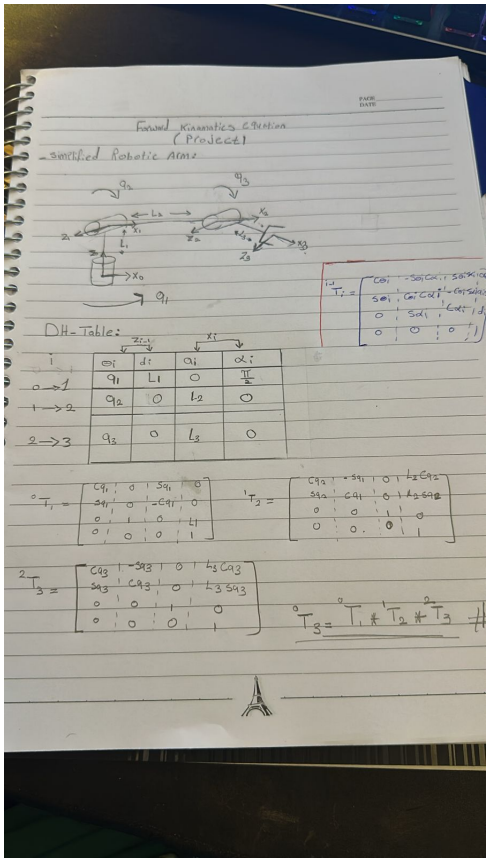

Figure 2.2: DH-convention by hand

2.3.1 DH-Parameter Table

The Denavit-Hartenberg (DH) table specifies the key parameters needed to describe the transformations between consecutive links of a robotic manipulator. These parameters capture the relative positions and orientations of adjacent joints, enabling efficient kinematic modeling.

i	θ_i	d_i	a_i	α_i
1	$\theta_1 = q_1$	L_1	0	$\frac{\pi}{2}$
2	$\theta_2 = q_2$	0	L_2	0
3	$\theta_3 = q_3$	0	L_3	0

Table 2.1: Denavit-Hartenberg Parameters

2.3.2 Transformation Matrices

Based on the DH-Table, the individual transformation matrices between the frames can be defined as follows:

- **Transformation matrix from frame 0 to frame 1 ($^0T_1$):**

$$^0T_1 = \begin{bmatrix} \cos(q_1) & -\sin(q_1) & 0 & 0 \\ \sin(q_1) & \cos(q_1) & 0 & 0 \\ 0 & 0 & 1 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.1}$$

- **Transformation matrix from frame 1 to frame 2 ($^1T_2$):**

$$^1T_2 = \begin{bmatrix} \cos(q_2) & -\sin(q_2) & 0 & L_2\cos(q_2) \\ \sin(q_2) & \cos(q_2) & 0 & L_2\sin(q_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.2}$$

- **Transformation matrix from frame 2 to frame 3 ($^2T_3$):**

$$^2T_3 = \begin{bmatrix} \cos(q_3) & -\sin(q_3) & 0 & L_3\cos(q_3) \\ \sin(q_3) & \cos(q_3) & 0 & L_3\sin(q_3) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.3}$$

- **Final Transformation Matrix ($^0T_3$):**

$$^0T_3 = {}^0T_1 \times {}^1T_2 \times {}^2T_3 \tag{2.4}$$

- **In the Matrix form**:

$$
{}^{0}T_{3} = \begin{bmatrix} \cos(q_1+q_2+q_3) & -\sin(q_1+q_2+q_3) & 0 & L_1 + L_2\cos(q_2) + L_3\cos(q_3) \\ \sin(q_1+q_2+q_3) & \cos(q_1+q_2+q_3) & 0 & L_2\sin(q_2) + L_3\sin(q_3) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.5}
$$

2.4 MATLAB/SIMULINK & Coppeliasim

2.4.1 SIMULINK Simscape Multibody

The process of converting a SolidWorks model into Simscape involves several steps that allow the physical components designed in SolidWorks to be utilized within MATLAB/Simulink for dynamic simulations. First, the SolidWorks CAD model is exported using the Simscape Multibody Link plug-in, which generates an XML file and associated physical geometry files. These files are then imported into Simscape using the Simscape Multibody Import tool. This conversion translates the SolidWorks geometrical and mechanical properties, such as masses, inertias, and joint constraints, into equivalent Simscape blocks. Each SolidWorks component, such as links and joints, becomes a Simscape block with defined physical connections. The resulting model is then connected in Simulink with additional control elements, such as motors or sensors, to create a complete simulation environment. As shown in Figure 4.2, the imported Simscape model of a robotic arm consists of multiple rigid bodies (links) and joints, along with motor actuation to drive the movements of the arm.

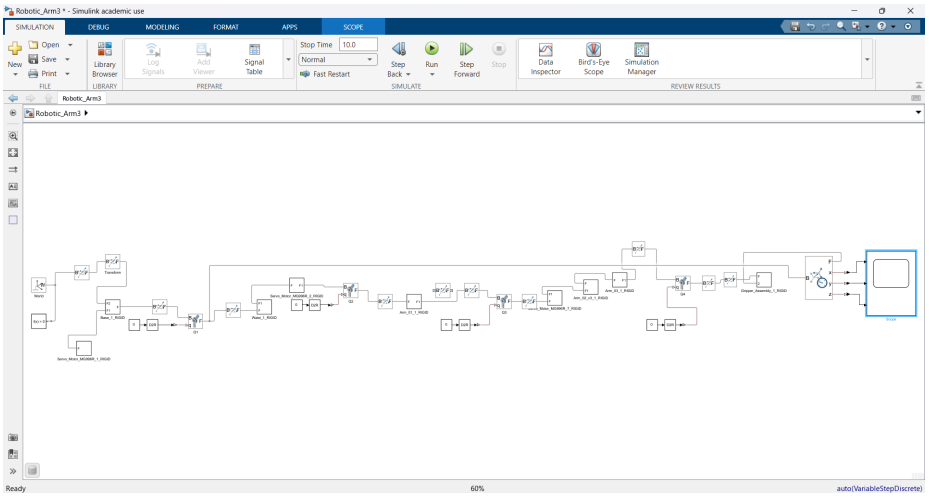

Figure 2.3: Simscape Model of Robotic Arm Imported from SolidWorks

The robotic arm in the image is a 3D model that was originally designed in SolidWorks and then converted to Simscape Mechanics for simulation. Simscape Mechanics is a MATLAB-based simulation environment that allows engineers to model and simulate mechanical systems. By converting the SolidWorks model to Simscape Mechanics, the engineer can analyze the arm's kinematics, dynamics, and control systems. The simulation can help to identify potential issues with the arm's design and optimize its performance before it is built and tested in the real world.

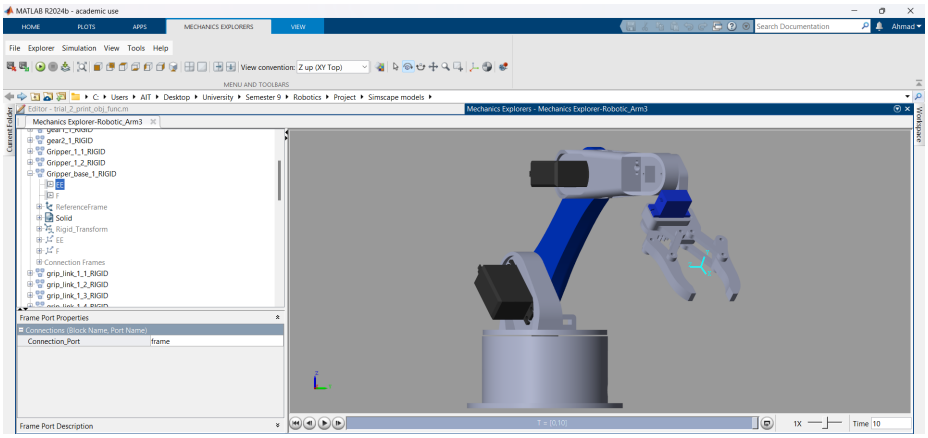

Figure 2.4: Simscape Model of Robotic Arm Imported from SolidWorks

2.4.2 MATLAB Code

The DH convention is a standardized method for representing the position and orientation of each link in a robot using four parameters: the link length, the twist angle, the offset distance, and the joint angle. The code uses these parameters to calculate the homogeneous transformation matrix for each link, which represents the link's position and orientation relative to the base frame. By multiplying the transformation matrices for all the links, the code can determine the position and orientation of the end effector relative to the base frame.

2.4.3 CoppeliaSim

The robotic arm shown in the image is a 3D model that was originally designed in SolidWorks and then exported to CoppeliaSim for simulation. CoppeliaSim is a powerful robotics simulation platform that allows users to test and evaluate the performance of robots in a virtual environment. By exporting the SolidWorks model to CoppeliaSim, we were able to simulate the arm's movements, interactions with its environment, and control systems.

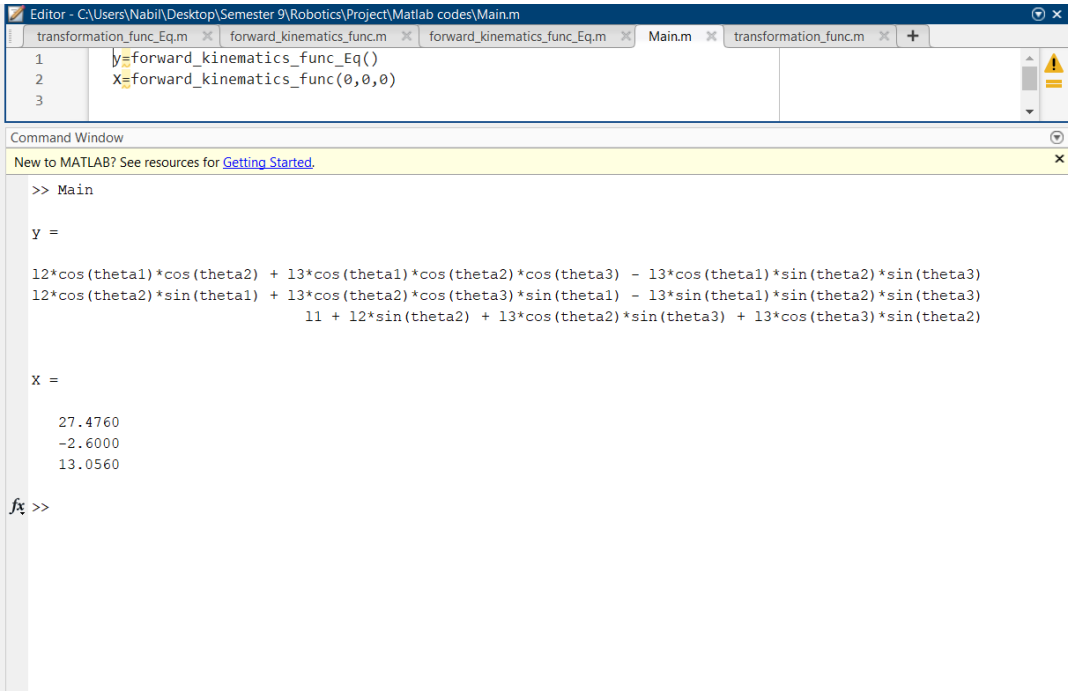

Figure 2.5: Matlab code

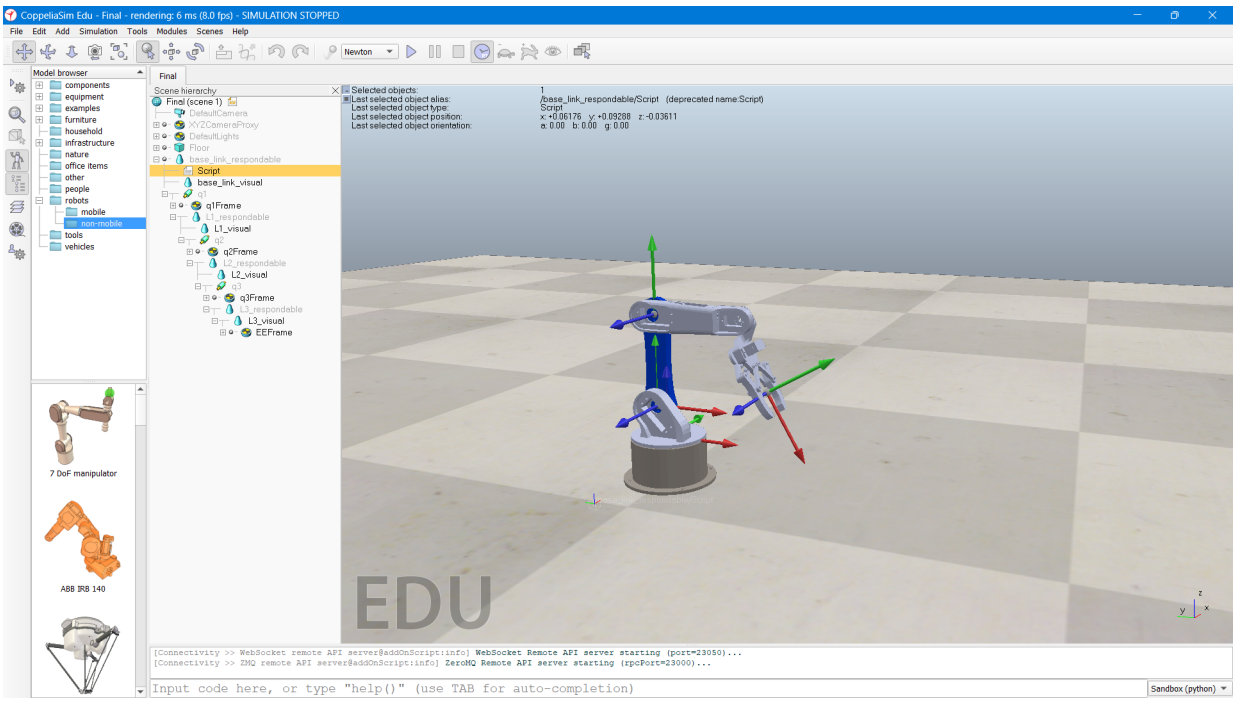

Figure 2.6: Coppeliasim simulation

Chapter 3

Kinematics

Forward Position Kinematics Using DH Parameters

The provided code calculates the forward kinematics of a 3-joint robotic arm, which is the process of determining the position of the end-effector (final segment) in 3D space based on the joint angles and link lengths. This approach uses the Denavit-Hartenberg (DH) convention, which standardizes parameters for describing the geometry of robotic manipulators.

Step-by-Step Explanation

1. **Input Joint Angles**:
The joint angles for each of the three joints are provided as inputs: θ_{11}, θ_{12}, and θ_{13} (in degrees). These angles define the rotation of each joint, where each rotation angle corresponds to a transformation matrix that locates each link relative to the previous one.
2. **Robot Parameters**:
The lengths of each link are predefined:

$$l_1 = 4.5\,\text{cm}, \quad l_2 = 12.1\,\text{cm}, \quad l_3 = 12.05\,\text{cm}$$

3. **Angle Conversion**:
The input angles are converted from degrees to radians, which is necessary for trigonometric operations in the transformation matrices.
4. **Transformation Matrices Using DH Convention**:
Each joint has an associated transformation matrix, calculated according to DH parameters. The DH parameters for each link are as follows:

- **Joint 1 Transformation Matrix (T_1)**:

$$\begin{aligned}&\text{Rotation angle: } \theta_1\\&\text{Link offset: } d = l_1\\&\text{Link length: } a = 1.374\\&\text{Link twist: } \alpha = \frac{\pi}{2}\end{aligned}$$

- **Joint 2 Transformation Matrix (T_2)**:

$$\begin{aligned}&\text{Rotation angle: } \theta_2\\&\text{Link offset: } d = -1.952\\&\text{Link length: } a = l_2\\&\text{Link twist: } \alpha = 0\end{aligned}$$

- **Joint 3 Transformation Matrix (T_3)**:

$$\begin{aligned}&\text{Rotation angle: } \theta_3\\&\text{Link offset: } d = -1.241\\&\text{Link length: } a = l_3 + 3.654\\&\text{Link twist: } \alpha = 0\end{aligned}$$

Each transformation matrix T is calculated using the DH formula, where:

$$T = \begin{bmatrix} \cos(\theta) & -\sin(\theta)\cos(\alpha) & \sin(\theta)\sin(\alpha) & a\cos(\theta) \\ \sin(\theta) & \cos(\theta)\cos(\alpha) & -\cos(\theta)\sin(\alpha) & a\sin(\theta) \\ 0 & \sin(\alpha) & \cos(\alpha) & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5. **Total Transformation Matrix**:
To find the final position of the end-effector, the individual transformation matrices are multiplied:

$$T_{\text{total}} = T_1 \cdot T_2 \cdot T_3$$

6. **Extracting End-Effector Position**:
The position of the end-effector in 3D space, denoted as X, is extracted from the first three rows of the fourth column of the total transformation matrix T_{total}. This vector $X = [X, Y, Z]^T$ represents the coordinates of the end-effector relative to the base frame.

Inverse Position Kinematics Using Newton-Raphson Method

Inverse kinematics (IK) involves determining the joint angles required to reach a specified position of the robotic arm's end-effector. Here, the Newton-Raphson method, a numerical approach, is used to iteratively calculate the joint angles that bring the end-effector to a target position in 3D space.

Step-by-Step Explanation

1. **Input Desired End-Effector Position**:
The desired position of the end-effector is provided as inputs X_e, Y_e, and Z_e, representing the target coordinates that the end-effector should reach.
2. **Setting Up Joint Angle Variables**:
Symbolic variables for the joint angles θ_1, θ_2, and θ_3 are defined. These angles are the unknowns we need to solve for to reach the target end-effector position.
3. **Newton-Raphson Method Parameters**:

- **Maximum Iterations**: Set to 500 to avoid an infinite loop if convergence cannot be achieved.
- **Tolerance**: Set to 1×10^{-3} to specify the acceptable margin of error for convergence.

4. **Initial Guess for Joint Angles**:
An initial guess of the joint angles is set to $[20°, 10°, 30°]$. This guess helps start the iterative process.
5. **Forward Kinematics Equation Setup**:
The forward kinematics function is defined using the DH parameters for the robot arm. This function calculates the position of the end-effector based on current joint angles and is used to evaluate the difference between the current and target positions.
6. **Error Function Definition**:
The error function represents the difference between the calculated end-effector position $[X, Y, Z]$ and the desired position $[X_e, Y_e, Z_e]$. This error function is given by:

$$f_{\text{vector}} = \begin{bmatrix} X - X_e \\ Y - Y_e \\ Z - Z_e \end{bmatrix}$$

Reducing the values in this error function to zero implies that the end-effector is at the target position.

7. **Jacobian Matrix**:

The Jacobian matrix J is computed symbolically with respect to the joint angles θ_1, θ_2, and θ_3. The Jacobian provides a relationship between small changes in joint angles and the corresponding changes in the position of the end effector.

8. **Iterative Newton-Raphson Update**:

In each iteration:

- The current joint angles are substituted into the error function and Jacobian matrix.
- The Newton-Raphson method is applied, updating the joint angles using:

$$\Delta q = -J^{-1} f_{\text{vector}}$$

  where Δq represents the angle adjustments needed to reduce the error. This step is crucial for steering the end-effector closer to the target position.

- The joint angles are updated by adding Δq to the current guess.

9. **Convergence Check**:

The magnitude of the angle updates (Δq) is checked against the tolerance. If it is below the tolerance, the method converges, and the loop terminates, returning the final joint angles.

10. **Result**:

If convergence is achieved within the maximum number of iterations, the joint angles $[q_1, q_2, q_3]$ that position the end-effector at the target coordinates are returned. If the solution does not converge, a warning is issued, indicating that the desired position might be out of reach or that a different initial guess may be needed.

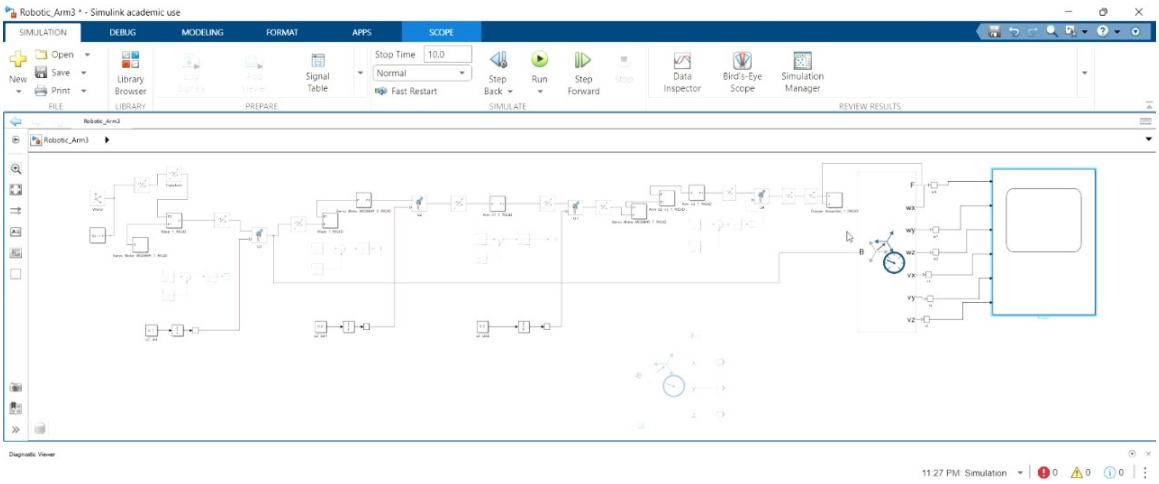

Figure 3.1: simscape model of position kinematics

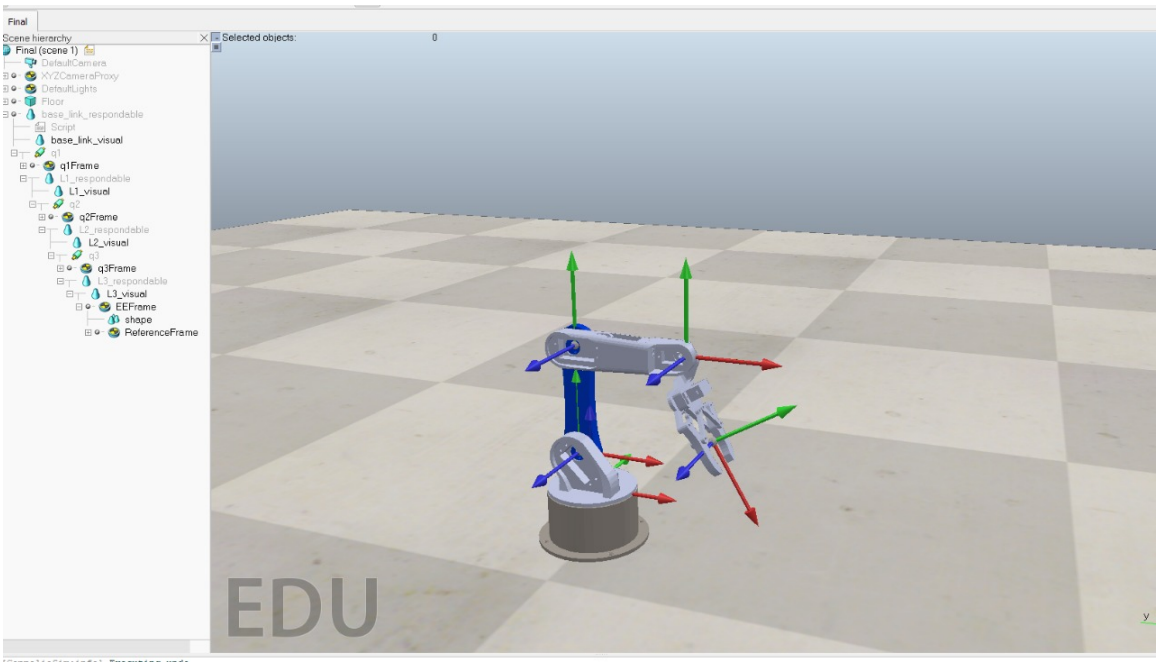

Figure 3.2: Coppeliasim initial position

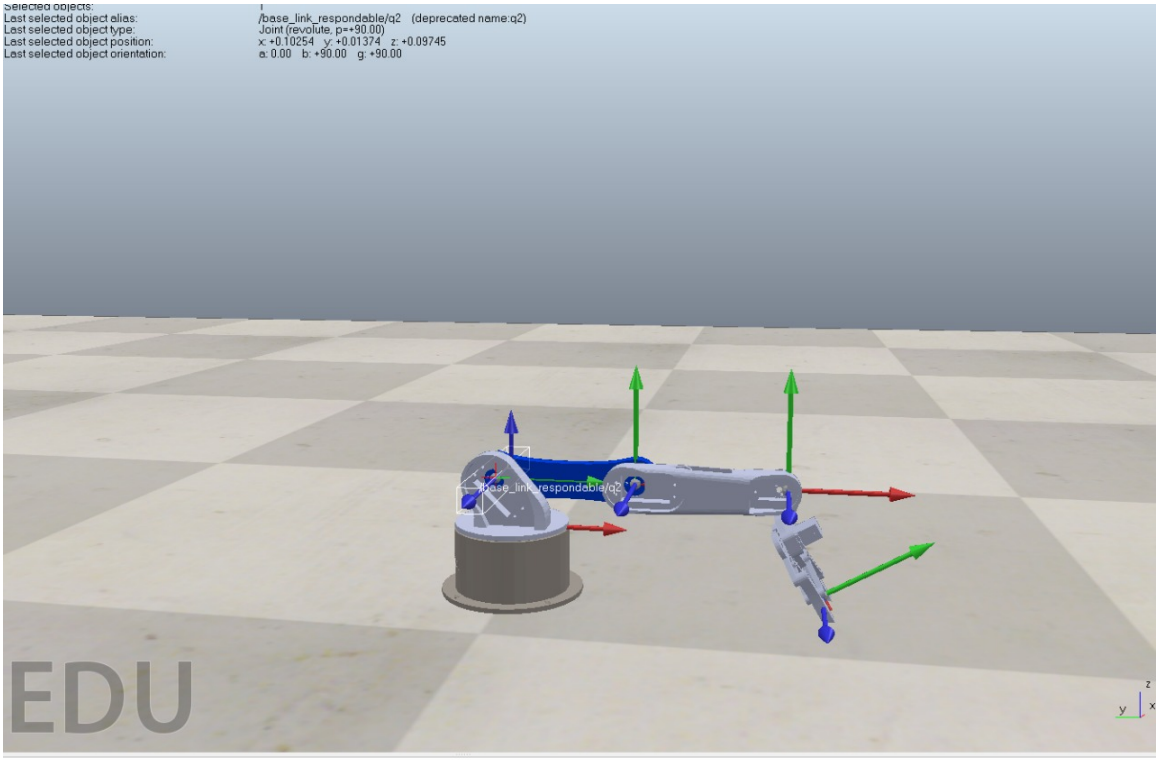

Figure 3.3: Coppeliasim zero position after using angles provided from python code

Forward Velocity Kinematics Using General Approach

Forward velocity kinematics is the process of calculating the velocity of the end effector based on the velocities of each joint. This calculation involves using the Jacobian matrix to map joint velocities to end-effector velocities. The general approach involves setting up a Jacobian that accounts for both the linear and angular components of motion. In this code, the Jacobian matrix is constructed symbolically and then numerically evaluated on the basis of the specified joint angles and velocities.

Step-by-Step Explanation

1. **Define Joint Variables and Link Lengths**:
Symbolic variables are defined for the joint angles θ_1, θ_2, θ_3 and their derivatives $\dot{\theta}_1$, $\dot{\theta}_2$, $\dot{\theta}_3$. The lengths of each link (l_1, l_2, l_3) are provided, as they are required to calculate the transformation matrices.
2. **Transformation Matrices**:
Using Denavit-Hartenberg (DH) parameters, transformation matrices are computed for each joint. These matrices T_1, T_2, and T_3 describe the position and orientation

of each link relative to the previous one. Each transformation matrix is based on the joint angle (θ_i) and the lengths of the links.

3. **Rotational Axes of Joints**:

- The rotation axis of the first joint is along the z-axis of the base frame, represented as $[0; 0; 1]$.
- The rotation axis of the second joint is derived from the z-axis of the transformation matrix T_1.
- The rotation axis of the third joint is derived from the z-axis of T_2.

These axes, Jw_1, Jw_2, and Jw_3, are used to compute the rotational part of the Jacobian.

4. **End-Effector Position Calculation**:

The position of the end-effector in terms of x, y, and z coordinates is obtained through the forward kinematics function 'forward_kinematics_func_Eq', which is derived using the DH parameters.

5. **Positional Jacobian (Linear Velocity Component)**:

The linear part of the Jacobian, J_v, is calculated by taking the partial derivatives of the end-effector's position $[f_x, f_y, f_z]$ with respect to each joint angle. This gives the positional Jacobian matrix, which maps joint velocities to linear velocities at the end-effector.

6. **Rotational Jacobian (Angular Velocity Component)**:

The rotational Jacobian, represented as J_w, is constructed using the rotational axes of each joint $[Jw_1, Jw_2, Jw_3]$. This matrix represents the contribution of the angular velocity of each joint to the angular velocity of the end effector.

7. **Construct the Full Jacobian Matrix**:

The full Jacobian matrix J_{sym} is formed by stacking the positional Jacobian J_v and the rotational Jacobian J_w vertically:

$$J_{\text{sym}} = \begin{bmatrix} J_v \\ J_w \end{bmatrix}$$

This combined Jacobian provides the relationship between joint velocities and linear and angular velocities of the end effector.

The Jacobian matrix **J** for a three-joint robotic manipulator can be written as:

$$\mathbf{J} = \begin{bmatrix} \mathbf{J_v} \\ \mathbf{J_w} \end{bmatrix}$$

where:

$$\mathbf{J_v} = \begin{bmatrix} \frac{\partial f_1}{\partial \theta_1} & \frac{\partial f_1}{\partial \theta_2} & \frac{\partial f_1}{\partial \theta_3} \\ \frac{\partial f_2}{\partial \theta_1} & \frac{\partial f_2}{\partial \theta_2} & \frac{\partial f_2}{\partial \theta_3} \\ \frac{\partial f_3}{\partial \theta_1} & \frac{\partial f_3}{\partial \theta_2} & \frac{\partial f_3}{\partial \theta_3} \end{bmatrix}$$

Here, f_1, f_2, f_3 are the x-, y-, and z-coordinates of the end-effector, respectively. These are derived from the forward kinematics of the manipulator.
The rotational Jacobian $\mathbf{J_w}$ is given by:

$$\mathbf{J_w} = \begin{bmatrix} \mathbf{J_{w1}} & \mathbf{J_{w2}} & \mathbf{J_{w3}} \end{bmatrix}$$

where:

$$\mathbf{J_{w1}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{J_{w2}} = T_1[1:3,3], \quad \mathbf{J_{w3}} = T_2[1:3,3]$$

Thus, the complete Jacobian matrix becomes:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial \theta_1} & \frac{\partial f_1}{\partial \theta_2} & \frac{\partial f_1}{\partial \theta_3} \\ \frac{\partial f_2}{\partial \theta_1} & \frac{\partial f_2}{\partial \theta_2} & \frac{\partial f_2}{\partial \theta_3} \\ \frac{\partial f_3}{\partial \theta_1} & \frac{\partial f_3}{\partial \theta_2} & \frac{\partial f_3}{\partial \theta_3} \\ \mathbf{J_{w1}} & \mathbf{J_{w2}} & \mathbf{J_{w3}} \end{bmatrix}$$

8. **Calculate Symbolic End-Effector Velocity**:
The symbolic end-effector velocity V_{sym} is computed by multiplying J_{sym} by the vector of joint angle velocities $[\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3]$.
9. **Evaluate the Jacobian Matrix Numerically**:
The symbolic Jacobian matrix is numerically evaluated at the specified joint angles q_1, q_2, and q_3, resulting in J_{num}, which is used in the final velocity calculation.
10. **Compute the Numeric End-Effector Velocity**:
Finally, the numeric velocity of the end effector V is calculated by substituting the joint angles (q_1, q_2, q_3) and joint velocities $(q_1\_dot, q_2\_dot, q_3\_dot)$ into V_{sym}.

Inverse Velocity Kinematics

Inverse velocity kinematics determines the required joint velocities ($\dot{q}$) needed to achieve a desired end-effector velocity (V_{desired}). In this process, we use the Jacobian matrix to map the desired end-effector velocity back to the joint velocities. However, because the Jacobian may not be directly invertible (especially for redundant or constrained robotic systems), we often use a pseudoinverse to find the solution.

Step-by-Step Explanation

1. **Define Joint Variables and Link Lengths**:
The symbolic variables for joint angles θ_1, θ_2, and θ_3 are defined. Link lengths (l_1, l_2, and l_3) are specified, as these are used in calculating the transformation matrices for each joint.
2. **Transformation Matrices**:
Each transformation matrix T_1, T_2, and T_3 is derived using Denavit-Hartenberg (DH) parameters. These matrices describe the position and orientation of each link relative to the previous one, considering the specified joint angles and link dimensions.
3. **Positional Jacobian (Linear Velocity Component)**:
The function 'forward_kinematics_func_Eq()' is used to get the end-effector's position in terms of the joint angles. By differentiating this position vector with respect to each joint angle (θ_1, θ_2, θ_3), the positional Jacobian J_v is computed. This represents the linear velocity component of the end-effector.
4. **Rotational Jacobian (Angular Velocity Component)**:

- The rotation axis of the first joint is along the z-axis of the base frame, represented as $[0; 0; 1]$.
- The rotation axis of the second joint is derived from the z-axis of the transformation matrix T_1.
- The rotation axis of the third joint is derived from the z-axis of T_2.

These axes Jw_1, Jw_2, and Jw_3 are combined to form J_w, the rotational Jacobian.
5. **Complete Jacobian Matrix**:
The full Jacobian J is constructed by combining J_v (for linear velocity) and J_w (for angular velocity):

$$J = \begin{bmatrix} J_v \\ J_w \end{bmatrix}$$

This complete Jacobian represents how joint velocities influence both the linear and angular velocity components of the end-effector.

6. **Substitute Joint Angles and Compute Numeric Jacobian**:
The symbolic Jacobian matrix J is evaluated at the specified joint angles (q_1, q_2, q_3) to obtain a numerical Jacobian matrix J_{num}.

7. **Calculate the Pseudoinverse of the Jacobian**:
Since the Jacobian may not be directly invertible due to of the under-actuation of our system, the pseudoinverse is calculated using:

$$\text{Pseudo}_J = (J_{\text{num}}^T J_{\text{num}})^{-1} J_{\text{num}}^T$$

This pseudoinverse provides the best approximation of joint velocities needed to achieve the desired end-effector velocity.

8. **Compute Joint Velocities**:
Finally, the joint velocities $\dot{q}$ are obtained by multiplying the pseudoinverse of the Jacobian with the desired end-effector velocity V_{desired}:

$$\dot{q} = \text{Pseudo}_J \times V_{\text{desired}}$$

This result gives the joint velocities required to produce the specified linear and angular velocities at the end-effector.

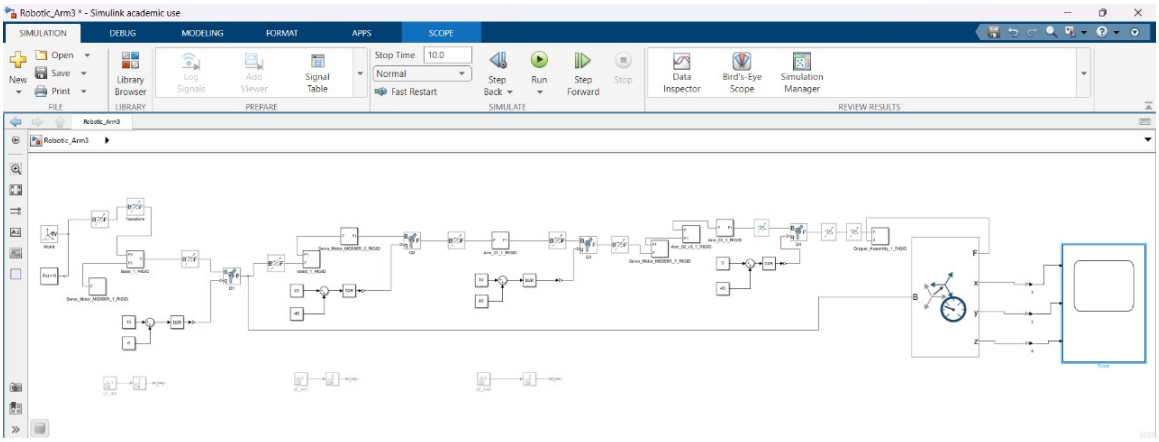

Figure 3.4: simscape model of velocity kinematics

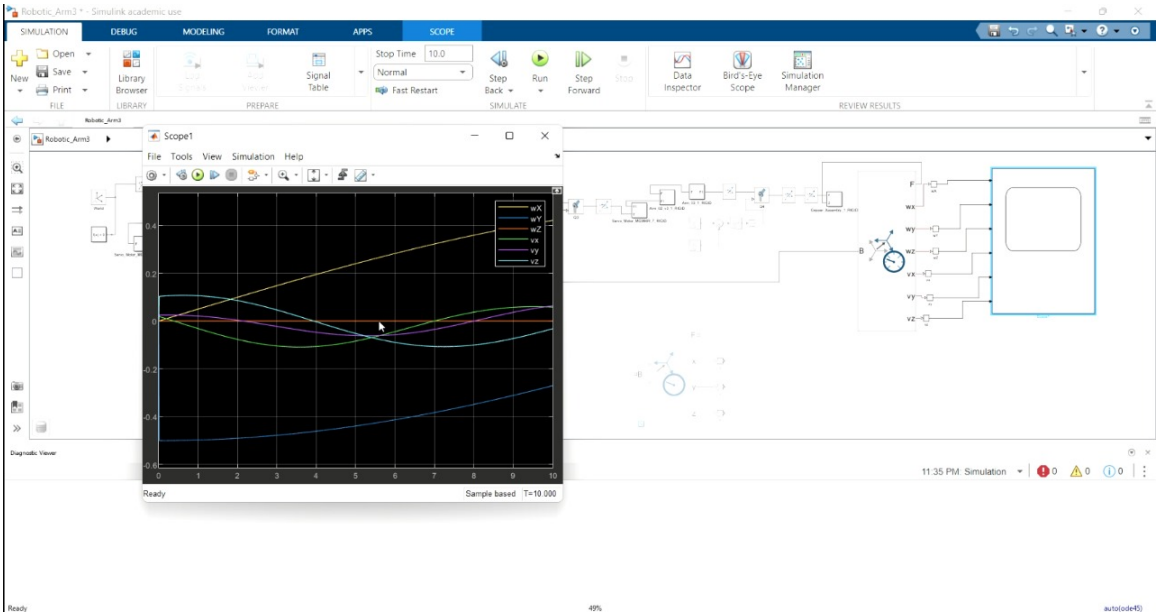

Figure 3.5: graph of inverse velocity kinematics

3.1 Forward Kinematics

The forward kinematics function computes the position of the end-effector given the joint angles. It uses the Denavit-Hartenberg (DH) parameters to construct the transformation matrices for each joint, and then multiplies them to obtain the final position of the end-effector.

3.1.1 Code

```
function X = forward_kinematics_func_Eq()
    % Define symbolic joint angles and link lengths
    syms theta1 theta2 theta3
    l1 = 4.5; l2 = 12.1; l3 = 12.05;

    % Transformation matrices using DH parameters
    T1 = transformation_func(theta1, l1, 1.374, pi/2);
    T2 = transformation_func(theta2, -1.952, l2, 0);
    T3 = transformation_func(theta3, -1.241, l3+3.654, 0);

    % Total transformation from base to end-effector
    T_total = T1 * T2 * T3;
    X = T_total(1:3, 4); % Extract the position of the end-
        effector
end

```

```
function T = transformation_func(theta, d, a, alpha)
    % Define the transformation matrix for given DH parameters
    T = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(
        alpha), a*cos(theta);
         sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(
            alpha), a*sin(theta);
         0,           sin(alpha),             cos(alpha),
                       d;
         0,           0,                      0,
                                  1];
end
```

Listing 3.1: Forward Kinematics Code

3.1.2 Explanation

The function `forward_kinematics_func_Eq` computes the end-effector's position X based on the given joint angles $\theta_1, \theta_2, \theta_3$. The function `transformation_func` generates the transformation matrix for each link using the Denavit-Hartenberg parameters. The final position is extracted from the resulting transformation matrix.

3.2 Forward Velocity Kinematics

The forward velocity kinematics function calculates the end-effector's velocity given the joint velocities. It derives the Jacobian matrix, which maps joint velocities to end-effector velocities.

3.2.1 Code

```
function [V_sym, V, J_sym, J_num] = forward_Velocity_func(q1, q2
    , q3, q1_dot, q2_dot, q3_dot)
    % Declare symbolic variables for joint angles and their
        derivatives
    syms theta1 theta2 theta3
    syms theta1_dot theta2_dot theta3_dot

    l1 = 4.5; % Length of link 1
    l2 = 12.1; % Length of link 2
    l3 = 12.05; % Length of link 3

    % Define transformation matrices for each joint based on DH
        parameters
```

```
11      T1 = transformation_func(theta1, l1, 1.374, pi/2); % First
            link
12      T2 = transformation_func(theta2, -1.952, l2, 0);   % Second
            link
13      T3 = transformation_func(theta3, -1.241, l3+3.654, 0);  %
            Third link
14
15      % Define rotational axes for each joint
16      Jw1 = [0; 0; 1];            % Base rotation axis for the
            first joint
17      Jw2 = T1(1:3, 3);           % Z-axis of first transformation
            for the second joint
18      Jw3 = T2(1:3, 3);           % Z-axis of second transformation
             for the third joint
19
20      % Set up symbolic vectors for angles and their derivatives
21      q = [theta1; theta2; theta3];
22      q_dot = [theta1_dot; theta2_dot; theta3_dot];
23
24      % Compute the forward kinematics to get end-effector
            position
25      X = forward_kinematics_func_Eq();
26      f1 = X(1); % x-coordinate of end-effector
27      f2 = X(2); % y-coordinate of end-effector
28      f3 = X(3); % z-coordinate of end-effector
29      fx = [f1; f2; f3]; % End-effector position vector
30
31      % Compute the symbolic positional Jacobian (Jv)
32      jv = jacobian(fx, q);
33
34      % Combine rotational Jacobians (Jw) into a single symbolic
            matrix
35      w = [Jw1, Jw2, Jw3]; % Rotational component Jacobians
36
37      % Construct the total Jacobian matrix (J_sym) symbolically
38      J_sym = [jv; w];
39
40      % Compute the symbolic end-effector velocity (V_sym) by
            multiplying J_sym with q_dot
41      V_sym = vpa(J_sym * q_dot);
42
43      % Evaluate the numeric Jacobian matrix (J_num) at the
            specified joint angles
44      J_num = double(subs(J_sym, [theta1, theta2, theta3], [q1, q2
            , q3]));
45
```

```
46      % Calculate the numeric end-effector velocity (V) using
            evaluated joint velocities
47      V = double(subs(V_sym, [theta1, theta2, theta3, theta1_dot,
            theta2_dot, theta3_dot], ...
48                      [q1, q2, q3, q1_dot, q2_dot, q3_dot]));
49  end
50
51  function T = transformation_func(theta, d, a, alpha)
52      % Define the transformation matrix based on the Denavit-
            Hartenberg parameters
53      T = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(
            alpha), a*cos(theta);
54           sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(
               alpha), a*sin(theta);
55           0,           sin(alpha),             cos(alpha),
                         d;
56           0,           0,                      0,
                                  1];
57  end
```

Listing 3.2: Forward Velocity Kinematics Code

3.2.2 Explanation

The function `forward_Velocity_func` computes the symbolic Jacobian matrix **J** and the end-effector velocity. The Jacobian matrix is computed by taking the derivative of the end-effector position with respect to the joint angles. The function then calculates the end-effector velocity by multiplying the Jacobian matrix with the joint velocities.

3.3 Inverse Kinematics

The inverse kinematics function uses the Newton-Raphson method to iteratively solve for the joint angles given a desired end-effector position. It calculates the error between the desired and current position and updates the joint angles accordingly.

3.3.1 Code

```
1   function [q1, q2, q3] = inverse_position_kinematics(Xe, Ye, Ze)
2       % Set up symbolic variables for joint angles
3       syms theta1 theta2 theta3
4
5       % Parameters for the Newton-Raphson method
```

```
6       max_iterations = 500;       % Maximum number of iterations
7       tolerance = 1e-3;           % Convergence tolerance for error
8
9       % Initial guess for joint angles (in degrees)
10      initial_guess = [20; 10; 30];
11      current_guess = initial_guess;
12
13      % Define forward kinematics using symbolic function
14      X = forward_kinematics_func_Eq();
15
16      % Desired end-effector positions
17      f1 = X(1) - Xe;
18      f2 = X(2) - Ye;
19      f3 = X(3) - Ze;
20      f_vector = [f1; f2; f3];        % Error function
21
22      % Jacobian matrix with respect to joint angles
23      J = jacobian(f_vector, [theta1; theta2; theta3]);
24
25      % Iterate until convergence or maximum iterations reached
26      for i = 1:max_iterations
27          % Current joint angle values in radians
28          theta1_val = deg2rad(current_guess(1));
29          theta2_val = deg2rad(current_guess(2));
30          theta3_val = deg2rad(current_guess(3));
31
32          % Substitute current joint values into error function
               and Jacobian
33          f_val = double(subs(f_vector, {theta1, theta2, theta3},
               {theta1_val, theta2_val, theta3_val}));
34          J_val = double(subs(J, {theta1, theta2, theta3}, {
               theta1_val, theta2_val, theta3_val}));
35
36          % Calculate joint velocity increment using inverse of
               Jacobian
37          delta_q = -inv(J_val) * f_val;
38
39          % Update current joint angles
40          current_guess = current_guess + delta_q';
41
42          % Check if error is below tolerance (convergence)
43          if norm(f_val) < tolerance
44              break;
45          end
46      end
47
```

```
48      % Return the final joint angles
49      q1 = current_guess(1);
50      q2 = current_guess(2);
51      q3 = current_guess(3);
52  end
```

Listing 3.3: Inverse Kinematics Code

3.3.2 Explanation

The function `inverse_position_kinematics` uses the Newton-Raphson method to solve for the joint angles that achieve the desired end-effector position. It iteratively updates the joint angles until the error (difference between current and desired position) is minimized.

3.4 Inverse Velocity Kinematics

The inverse velocity kinematics function computes the required joint velocities $(\dot{q}_1, \dot{q}_2, \dot{q}_3)$ to achieve a desired end-effector velocity. This is done by using the pseudo-inverse of the Jacobian matrix.

3.4.1 Code

```
1   function q_dot = inverse_velocity_func(V_desired, q1, q2, q3)
2       % Calculate the Jacobian matrix symbolically
3       syms theta1 theta2 theta3
4       l1 = 4.5; % Length of link 1
5       l2 = 12.1; % Length of link 2
6       l3 = 12.05; % Length of link 3
7
8       % Define transformation matrices for each joint based on DH
            parameters
9       T1 = transformation_func(theta1, l1, 1.374, pi/2); % First
            link
10      T2 = transformation_func(theta2, -1.952, l2, 0);    % Second
            link
11      T3 = transformation_func(theta3, -1.241, l3+3.654, 0);  %
            Third link
12
13      % Joint angle vector
14      q = [theta1; theta2; theta3];
15
16      % Forward kinematics to get the end-effector position
```

```
    X = forward_kinematics_func_Eq();
    f1 = X(1);
    f2 = X(2);
    f3 = X(3);
    fx = [f1; f2; f3]; % Position vector of the end-effector

    % Compute the positional Jacobian (Jv) by differentiating
       position with respect to joint angles
    Jv = jacobian(fx, q);

    % Rotation components of the Jacobian for each joint axis
    Jw1 = [0; 0; 1];
    Jw2 = T1(1:3, 3);
    Jw3 = T2(1:3, 3);
    Jw = [Jw1, Jw2, Jw3]; % Combine rotational components into
       Jw

    % Complete Jacobian by combining Jv and Jw
    J = [Jv; Jw];

    % Substitute actual joint angles into the Jacobian matrix
       for numerical evaluation
    J_num = double(subs(J, [theta1, theta2, theta3], [q1, q2, q3
       ]));

    Pseudo_J = inv(transpose(J_num)*J_num)*transpose(J_num);
    q_dot = Pseudo_J * V_desired;
end

function T = transformation_func(theta, d, a, alpha)
    % Define the transformation matrix using DH parameters
    T = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(
       alpha), a*cos(theta);
         sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(
            alpha), a*sin(theta);
         0,           sin(alpha),             cos(alpha),
                       d;
         0,           0,                      0,
                                  1];
end
```

Listing 3.4: Inverse Velocity Kinematics Code

3.4.2 Explanation

The function `inverse_velocity_func` calculates the joint velocities $\dot{q}_1, \dot{q}_2, \dot{q}_3$ required to achieve a desired end-effector velocity V_{desired}. This is done by computing the pseudo-inverse of the Jacobian matrix and multiplying it by the desired velocity vector.

3.5 Jacobian Matrix

The Jacobian matrix $\mathbf{J}$ relates joint velocities to the end-effector's linear and angular velocities. It is composed of two parts:

$$\mathbf{J} = \begin{bmatrix} \mathbf{J_v} \\ \mathbf{J_w} \end{bmatrix}$$

where: - $\mathbf{J_v}$ is the positional Jacobian, which maps joint velocities to linear velocities. - $\mathbf{J_w}$ is the rotational Jacobian, which maps joint velocities to angular velocities.

3.5.1 Jacobian Matrix in Code

```
function [J_sym, V_sym, J_num, V] = forward_Velocity_func(q1, q2
    , q3, q1_dot, q2_dot, q3_dot)
    % Same code as previous function for Jacobian and velocity
        calculation
```

Listing 3.5: Jacobian Matrix Code

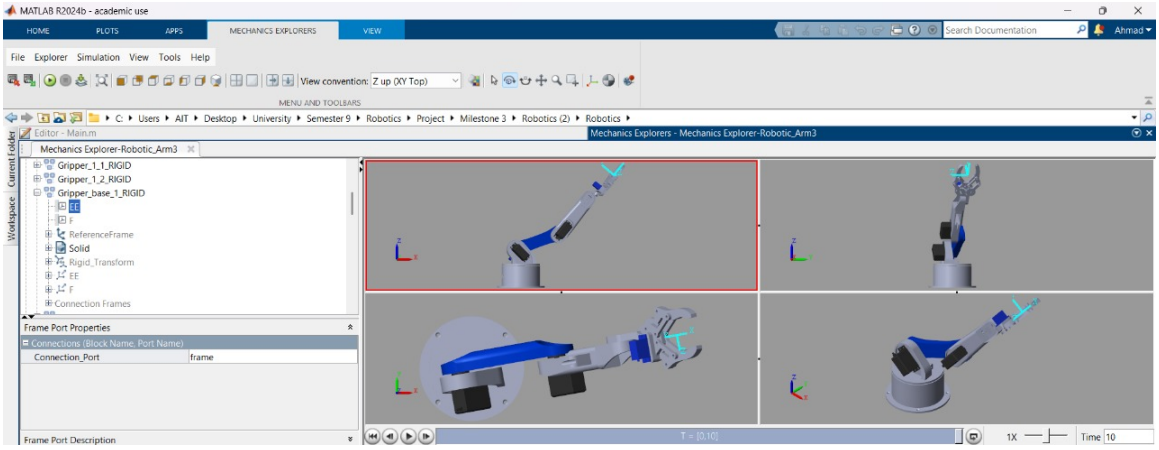

Figure 3.6: simscape model using both velocity and position kinematics

Chapter 4

Robot Hardware

The hardware of our robotic system was custom-designed and 3D printed, enabling precise manufacturing of the manipulator's joints, links, and end-effector. The 3D-printed components provide the necessary structure to achieve the required degrees of freedom for movement. Actuators, such as motors, drive these joints to execute specific tasks. A control unit (Arduino) processes the inputs and commands the actuators to ensure smooth operation. This 3D-printed approach allowed for rapid prototyping and ensured a tailored fit for the robotic design, enabling it to perform its designated functions effectively.

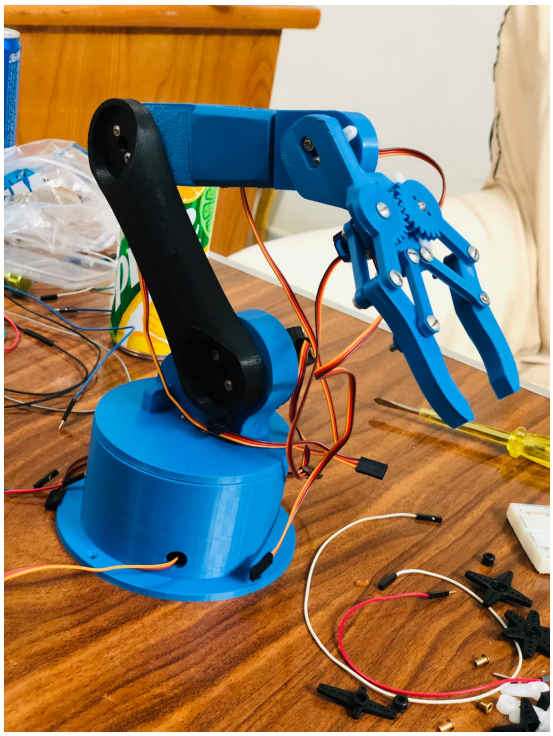

Figure 4.1: 3D printed prototype Robotic Arm

Figure 4.2: 3D printed final Robotic Arm

Chapter 5

Trajectory Planning

In this milestone, the target was to implement 2 trajectories in hardware and in simulation, in simscape and in coppeliaSim, comparing the performance between both trajectories, since we implemented a task space and a joint space trajectory, in the next sections, we show results from the implemented trajectories.

5.1 MATLAB & Simscape

In both the joint space and task space trajectories, the initial and final positions of the robot are kept the same, as well as the time of simulation and the time step.

5.1.1 Task Space Trajectory:

In the implemented task space trajectory, the required trajectory was a straight line, with $x_i = 33, y_i = 0$ and $z_i = 4.045$, which is the zero position of our robot, as shown in Figure 5.1, the time of simulation was used to be 10 seconds, with 0.1s per step, and the final position of the trajectory was used to be $x_f = 9.3630, y_f = 7.8565$ & $z_f = 30.9651$, we got these positions by inputting angles q_1, q_2 & q_3 to the forward kinematics code, to get the positions, then we ran the straight line trajectory code, Figure 5.3 shows the final orientation of the links in the MATLAB environment, with the straight line trajectory plotted in the 3D space as well, Figure 5.2 shows the final position of the robotic arm on Simscape.
In Figure 5.3, the straight line trajectory can be viewed in the 3D space, as well as the final orientation of the robotic arm, the trajectory, as expected showed a straight line behavior.
Running the same trajectory on Simscape from Matlab can be done by inputting the angle arrays into the joints, resulting in a simulated trajectory as expected and seen in the videos.

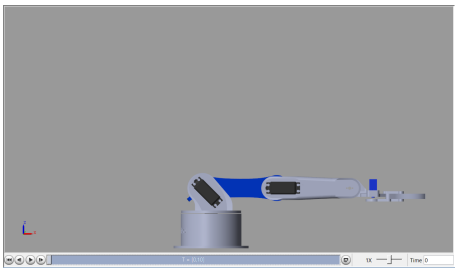

Figure 5.1: The robotic arm's zero position

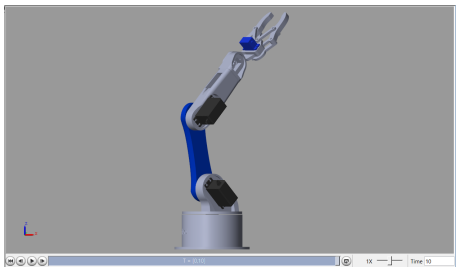

Figure 5.2: The robotic arm's final position

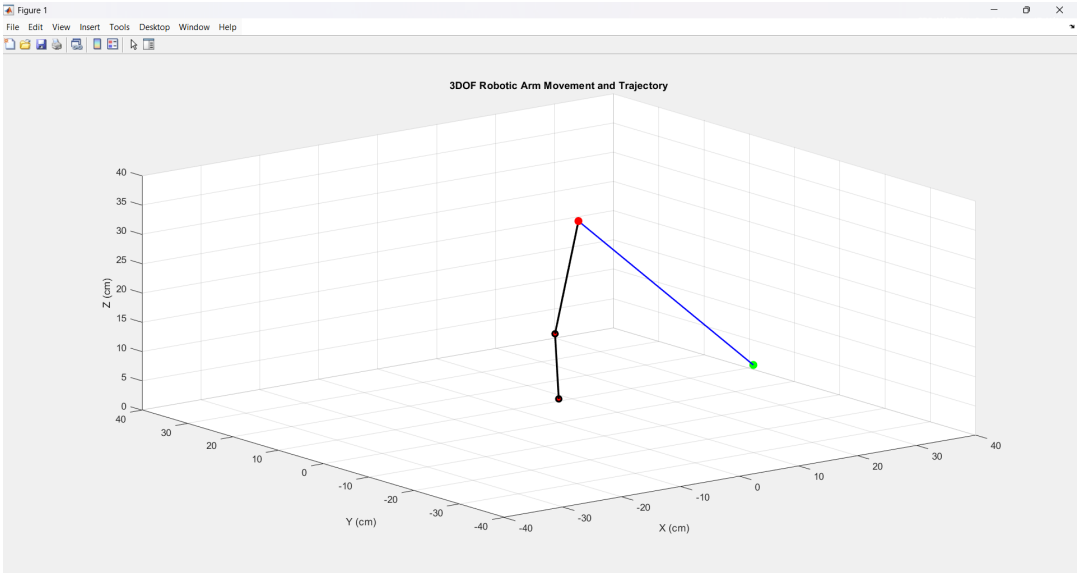

Figure 5.3: Task Scape Trajectory from MATLAB

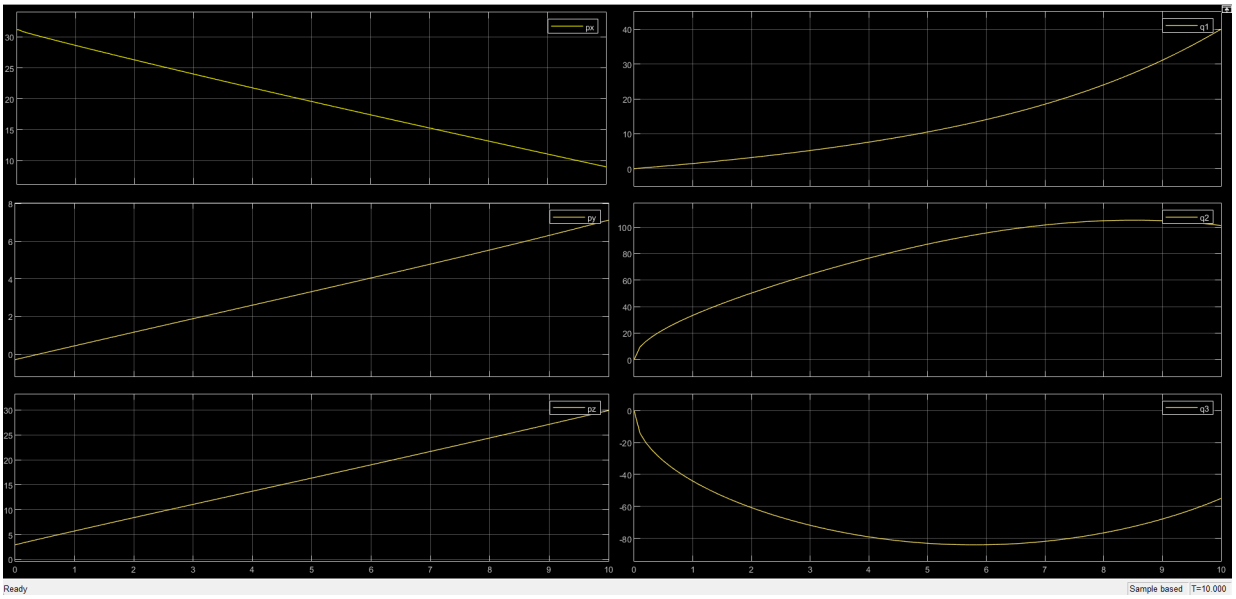

Figure 5.4: Task Space Trajectory Scope

Figure 5.4 plots the changes of angles q_1, q_2 and q_3 with time, as well as the changes of end effector positions x, y and z with time, the curves show that the variation in values is aggressive to ensure that the end effector moves in the desired trajectory, a straight line as shown in the example illustrated in figure 5.3.

5.1.2 Joint Space Trajectory:

In the implemented joint space trajectory, the same start and end positions, as well as the same time of simulation and time step was used to get the trajectory, we evaluated the polynomial expansion up to the 5th order, and the numeric equations of $q(t)$, $\dot{q}(t)$ and $\ddot{q}(t)$ for angles q_1, q_2 and q_3 are as follows:

$$q_1(t) = 0.0000 + 0.0000 * t + -0.0000 * t^2 + 0.4000 * t^3 + -0.0600 * t^4 + 0.0024 * t^5 \quad (5.1)$$

$$\dot{q_1}(t) = 0.0000 + -0.0000 * t + 1.2000 * t^2 + -0.2400 * t^3 + 0.0120 * t^4 \quad (5.2)$$

$$\ddot{q_1}(t) = -0.0000 + 2.4000 * t + -0.7200 * t^2 + 0.0480 * t^3 \quad (5.3)$$

$$q_2(t) = -0.0156 + -0.0000 * t + 0.0000 * t^2 + 1.0118 * t^3 + -0.1518 * t^4 + 0.0061 * t^5 \quad (5.4)$$

$$\dot{q_2}(t) = -0.0000 + 0.0000 * t + 3.0353 * t^2 + -0.6071 * t^3 + 0.0304 * t^4 \quad (5.5)$$

$$\ddot{q_2}(t) = 0.0000 + 6.0706 * t + -1.8212 * t^2 + 0.1214 * t^3 \quad (5.6)$$

$$q_3(t) = 0.0245 + 0.0000 * t + -0.0000 * t^2 + -0.5502 * t^3 + 0.0825 * t^4 + -0.0033 * t^5 \quad (5.7)$$

$$\dot{q_3}(t) = 0.0000 + -0.0000 * t + -1.6507 * t^2 + 0.3301 * t^3 + -0.0165 * t^4 \quad (5.8)$$

$$\ddot{q_3}(t) = -0.0000 + -3.3015 * t + 0.9904 * t^2 + -0.0660 * t^3 \quad (5.9)$$

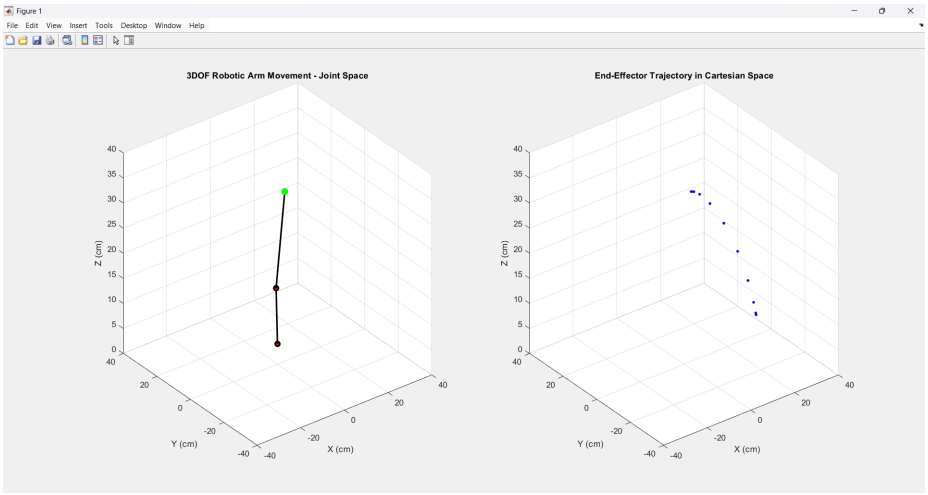

Figure 5.5: The final robot's arm (left), and the points of the trajectory (right)

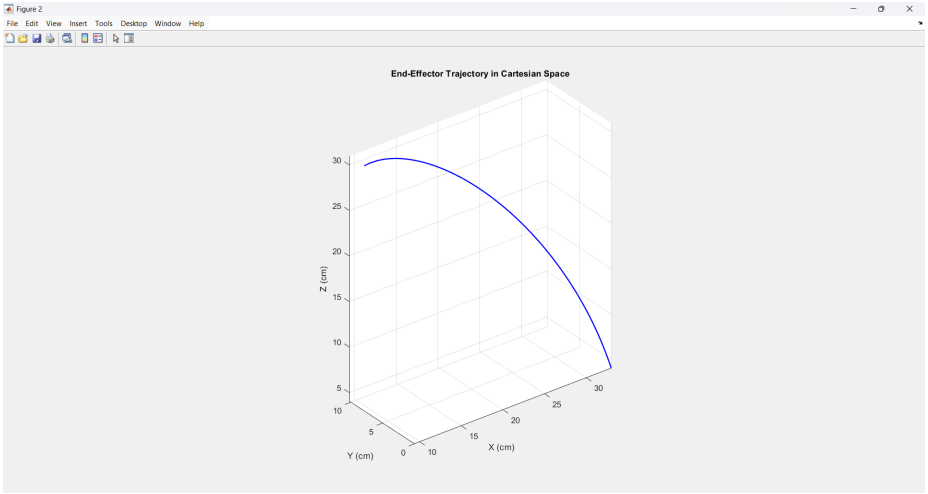

Figure 5.6: The joint space trajectory 3D plot

Running the code to showcase the output trajectory of the end position can be shown in figure 5.5. Connecting the points together can shows us a better insight on the output trajectory, figure 5.6 shows the trajectory.

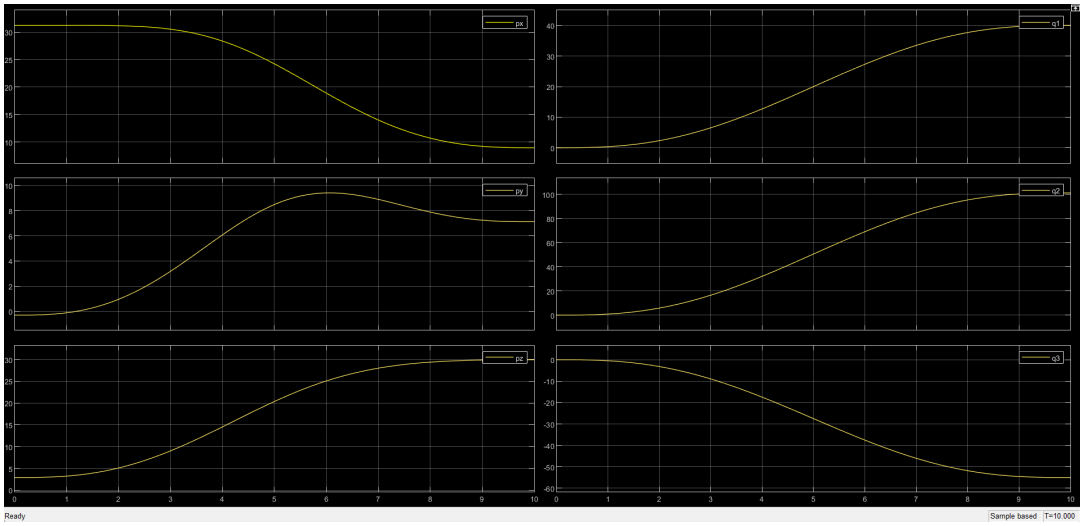

Figure 5.7: The joint space scope

We then ran the same trajectory on Simscape and Coppeliasim by using the angles arrays as input to the joints, Figure 5.7 plots the changes of angles q_1, q_2 and q_3 with time, as well as the changes of end effector positions x, y and z with time, the plots show much smoother variation in values when compared to the task space trajectory curves shown in figure 5.4, which allows the motors and joints to move much more freely compared to the task space.

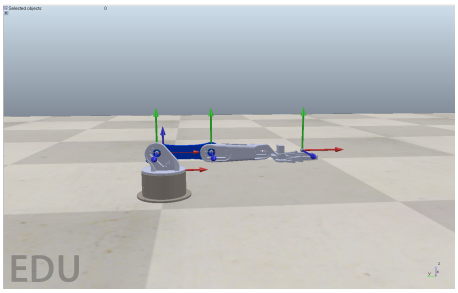

Figure 5.8: The robotic arm's zero position

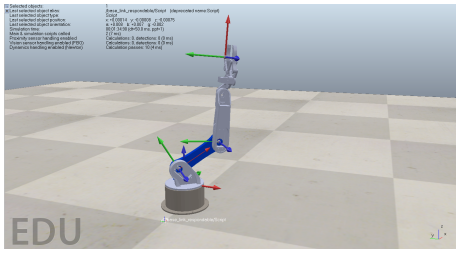

Figure 5.9: The robotic arm's final position

```
Time: 9.499999999999982, q1: 39.98633603154977, q2: 29.968272411225456, q3: 54.91652625412223
Time: 9.599999999999982, q1: 40.00859063321542, q2: 29.98495308344613, q3: 54.94708769717886
Time: 9.699999999999982, q1: 40.02237063278534, q2: 29.9952817183721 83, q3: 54.96601127435639
Time: 9.799999999999981, q1: 40.029591594901945, q2: 30.000694104717837, q3: 54.97592756159702
Time: 9.89999999999998, q1: 40.0323045548607, q2: 30.002727571616077, q3: 54.97965317181195
Time: 9.99999999999998, q1: 40.03269890096439, q2: 30.003023149051984, q3: 54.98019471311534
[sandboxScript:info] Simulation stopping...
[0.08309726503918399, 0.0698515550517485, 0.3096542932176652]
Warning: Maximum iterations reached without convergence.
Joint angles from IK at final position: q1 = 10.170212133886688, q2 = -34.64852750557793, q3 = 134.09671073056276
[sandboxScript:info] Simulation stopped.
```

Figure 5.10: CoppeliaSim code outputs

After executing the code in CoppeliaSim, the robot successfully follows the desired trajectory as specified in the variable 5.9. This trajectory is defined to guide the robot's end effector to a particular set of coordinates or pose in the simulation environment. During the simulation, the robot moves along the path defined by this trajectory, and its progress is tracked.

Additionally, the code outputs an array of joint angles corresponding to the robot's movements as it progresses along the trajectory. These joint angles are crucial for understanding how the robot's individual joints must move to achieve the specified end effector position. Along with the joint angles, the simulation also prints the actual position of the robot's end effector at each step, as shown in the visual output in 5.10.

Upon closer inspection of the results, the final position of the end effector—after the trajectory has been completed—closely matches the desired final position that was pre-defined for the trajectory. While there might be slight variations due to factors such as numerical precision or minor inaccuracies in the simulation, the end effector's position is effectively aligned with the intended target, demonstrating that the robot has moved as expected along the planned trajectory.